

Dokumentacja Projektu: Machine Learning w Klasyfikacji Elementów Gitarowych

Autor: kubuk224-cyber

Czerwiec 2026

Spis treści

1 Wstęp i Cel Projektu

Niniejszy projekt koncentruje się na wykorzystaniu głębokich sieci konwolucyjnych (CNN) do automatycznej klasyfikacji atrybutów gitar elektrycznych na podstawie zdjęć.

Rozpoznawanie takich detali jak kształt korpusu (Single-cut vs Double-cut), rodzaj przetworników, marka (na podstawie kształtu główki) oraz rodzaj zastosowanego mostka wymaga zróżnicowanego podejścia do architektury modeli, technik augmentacji oraz inżynierii zbiorów danych. W ramach projektu zaimplementowano proces uczenia transferowego (ang. *Transfer Learning*) z wykorzystaniem środowiska PyTorch.

2 Zbiór Danych (Dataset) i Augmentacja

2.1 Pozyskiwanie Danych

Zbiór danych został pozyskany m.in. poprzez autorski skrypt `scraper.py`, pobierający wysokiej rozdzielczości zdjęcia ze stron sklepów muzycznych (np. Thomann). Struktura katalogów została dostosowana do specyfiki problemu:

- Zdjęcia całych korpusów do klasyfikacji *cutaway* i elektroniki.
- Ciasne kadry na główki (headstock) do identyfikacji producenta.
- Zbliżenia na mostki do rozróżniania systemów (m.in. *Floyd Rose*, *Tremolo*, *Hardtail*, *Tune-o-matic*).

2.2 Autorska klasa Data Loadera

Z racji współdzielenia folderu `./data` przez zdjęcia z różnych podproblemów, zaimplementowano niestandardową klasę dziedziczącą po `torch.utils.data.Dataset` (np. `GuitarBridgesDataset`). Pozwoliło to na selektywne wczytywanie tylko dozwolonych klas (np. `ALLOWED_CLASSES`) i całkowite ignorowanie niechcianych podfolderów.

2.3 Transformacje i Augmentacja

Aby zapobiec zjawisku przeuczenia (overfittingu) przy ograniczonej liczbie zdjęć, zastosowano agresywną augmentację w locie (ang. *on-the-fly augmentation*) przy użyciu modułu `torchvision.transforms`:

- **RandomRotation:** Obrót o losowy kąt (od 15 do 30 stopni w zależności od modelu).
- **RandomHorizontalFlip:** Odbicia lustrzane (z prawdopodobieństwem 50%).
- **ColorJitter:** Zmiany kontrastu i jasności, uodparniające model na różne warunki oświetleniowe.
- **AddGaussianNoise:** Autorska klasa wstrzykująca szum Gaussa do tensora, symulująca gorszej jakości matryce fotograficzne.

Zbiór walidacyjny został pozbawiony powyższych transformacji (poza reskalowaniem do wymiarów wejściowych sieci oraz normalizacją do standardu ImageNet).

3 Architektura Sieci Neuronowych

Ze względu na różną ziarnistość detali, projekt wykorzystuje podejście wielomodelowe oparte o architekturę ResNet:

3.1 ResNet-18 dla makro-struktur

Do rozpoznawania kształtów korpusu (Single/Double cut), zastosowano lekką i szybką sieć **ResNet-18**. Główne kontury drewna są łatwo rozpoznawalne na wczesnych mapach cech, dlatego głębszy model stwarzałby jedynie niepotrzebne ryzyko szybkiego przeuczenia.

3.2 ResNet-50 dla mikro-detali

Przy zadaniach o wysokim stopniu trudności – takich jak rozpoznawanie rozmytego logo na główce gitary lub drobnych elementów mechanicznych (śruby blokujące we Floyd Rose w kontrze do standardowych siodełek we vibrato) – architektura została podmieniona na **ResNet-50**. Sieć ta generuje aż 2048 cech wyjściowych przed warstwą w pełni połączoną (Fully Connected), co pozwala na skrupulatną analizę małych fragmentów obrazu.

4 Proces Trenowania i Optymalizacja

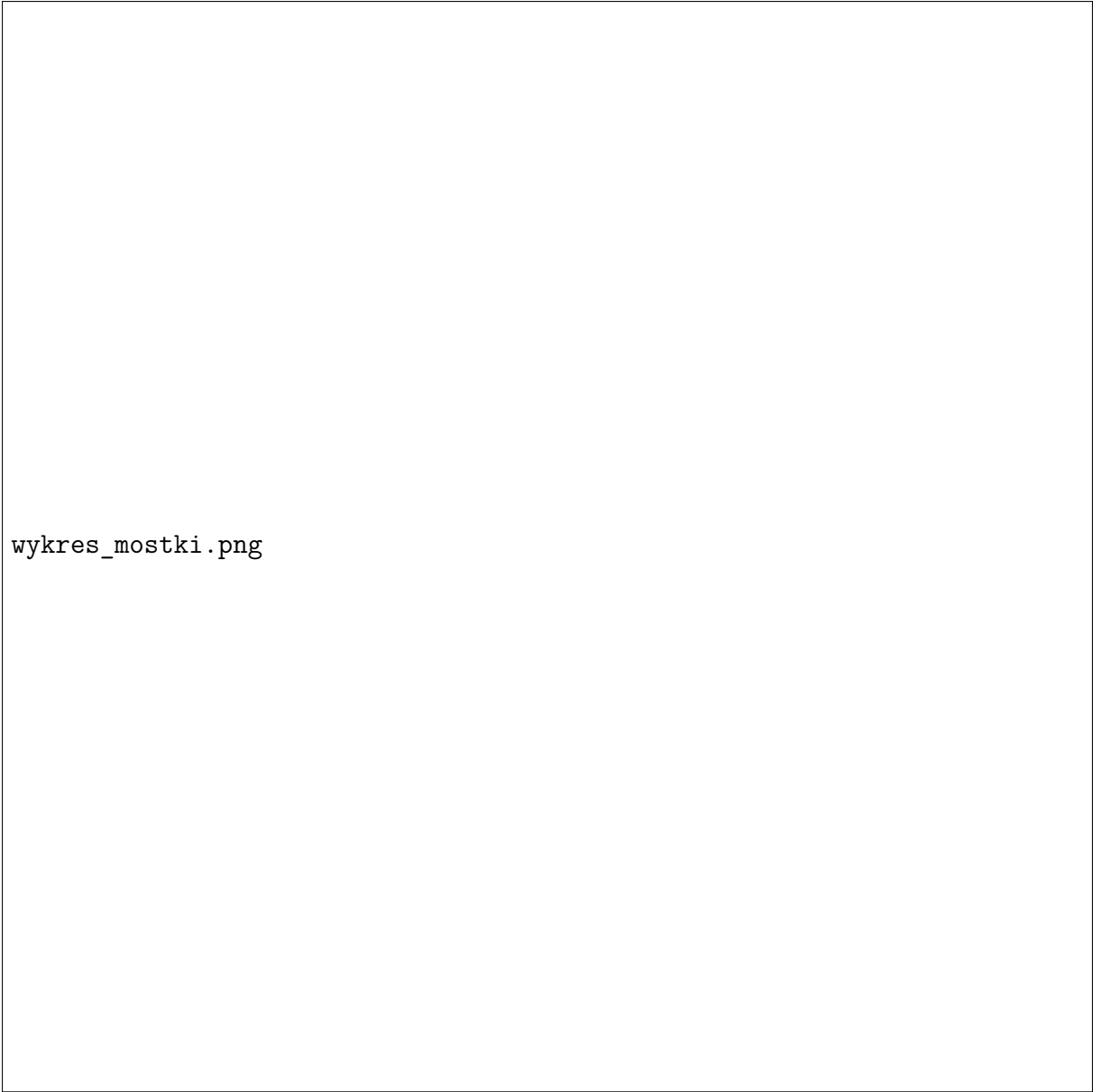
4.1 Metryki i Funkcje Stracy

Jako funkcję celu zastosowano standardową stratę krzyżowej entropii (**CrossEntropyLoss**).

Początkowe eksperymenty z optymalizatorem SGD (*Stochastic Gradient Descent*) zastąpiono nowoczesnym algorytmem **AdamW**. Połączenie adaptacyjnego tempa uczenia z ulepszoną regularyzacją wag (`weight_decay = 1e-3`) znacząco zapobiegało degradacji wyników na zbiorze walidacyjnym dla pojemnych modeli (ResNet-50).

4.2 Harmonogram Szybkości Uczenia (LR Scheduler)

Wdrożono technikę **CosineAnnealingLR**. Pozwala ona na dynamiczne dostosowywanie parametru *Learning Rate* zderzając szybkie poszukiwania ekstremum w pierwszych epokach z powolnym, płynnym (sinusoidalnym) wygaszaniem kroku w epokach końcowych, co pozwala "osiąść" w optymalnym minimum lokalnym.



wykres_mostki.png

Rysunek 1: Wykresy: Spadek straty (Loss), Dokładność (Accuracy) oraz krzywa Szybkości Uczenia (Learning Rate) podczas procesu trenowania ResNet-50.

5 Historia Rozwoju i Repo GitHub

Rzwoj projektu został w pełni zewidencjonowany w systemie kontroli wersji Git (autor: *kubuk224-cyber*). Poniższa tabela przedstawia kluczowe kamienie milowe projektu:

Data	Wiadomość Commita	Opis zmian
16 Maj 2026	<i>first work with guitar classifier</i>	Inicjalizacja środowiska i repozytorium. Podstawowy model.
16 Maj 2026	<i>increased epochs and increased dataset</i>	Rozbudowa początkowej bazy zdjęć i pierwsza runda strojenia hiperparametrów.
24 Maj 2026	<i>Changed into rasnet50 model)</i>	Ewolucja architektury z ResNet-18 na pojemniejszy model ResNet-50 dla większej skuteczności w detalach.
25 Maj 2026	<i>95% validation on 7 epochs</i>	Osiągnięcie bardzo satysfakcjonującej celności klasyfikacji we wczesnym etapie treningu.
1 Cze 2026	<i>more dataset</i>	Skalowanie danych - wdrożenie zdjęć o wyższej jakości w celu dalszej stabilizacji modelu.
5 Cze 2026	<i>added guitar bridge ML / added bridges</i>	Implementacja złożonego klasyfikatora mostków gitarowych ignorującego korpusy w locie (Custom Dataset).

Tabela 1: Kamienie milowe w rozwoju projektu klasyfikatora elementów gitarowych na podstawie repozytorium GitHub.

6 Wnioski

Zastosowanie architektury Pipeline (osobne modele dla korpusów i detali) pozwoliło wyeliminować problem utraty cech na skutek normalizacji obrazów wejściowych (kompresja do 224×224 px). Mechanizmy zabezpieczające takie jak sztuczny szum (Gaussian Noise), odpowiedni weight decay w AdamW oraz Cosine Annealing udowodniły swoją wartość osiągając nałożone cele projektu (powyżej 95% trafności na walidacji).